ROS 2 Service & Client

Complete Tutorial

Robot Grid Pathfinder

BFS-based shortest path finding over a grid with obstacles

What you will build:

A ROS 2 service that receives a start cell and an end cell on a 10x10 grid, navigates around obstacles using Breadth-First Search, and returns the shortest path step by step. The client visualises the result as an ASCII grid in the terminal.

Environment

Item	Version
ROS 2	Humble Hawksbill
OS	Ubuntu 22.04 LTS
Python	3.10+
Build Tool	colcon
Algorithm	Breadth-First Search (BFS)

Contents

1 Introduction to the Service-Client Model **3**
1.1 What This Tutorial Builds . . . 3

2 Why Two Separate Packages? **3**

3 The Grid and the Algorithm **4**
3.1 Grid Layout . . . 4
3.2 Obstacle Map . . . 4
3.3 Breadth-First Search (BFS) . . . 4

4 Create the Two Packages **5**
4.1 Navigate to workspace src . . . 5
4.2 Package 1 — Interface package . . . 5
4.3 Package 2 — Node package . . . 5
4.4 Expected workspace structure . . . 5

5 Set Up the Interface Package **6**
5.1 Create the srv folder and file . . . 6
5.2 Write the .srv definition . . . 6
5.3 Update CMakeLists.txt . . . 6
5.4 Update pathfinder_interfaces/package.xml . . . 7
5.5 Build and verify . . . 7

6 Set Up the Node Package **7**
6.1 Update pathfinder_nodes/package.xml . . . 7
6.2 Create the node files . . . 8

7 Write the Server Node **8**
7.1 Key Parts Explained . . . 11

8 Write the Client Node **11**

9 Update setup.py **14**

10 Build and Run **14**
10.1 Build both packages . . . 14
10.2 Terminal 1 — Start the server . . . 15
10.3 Terminal 2 — Run the client . . . 15

11 Test Cases **15**
11.1 Test 1 — Standard path (original grid example) . . . 15
11.2 Test 2 — Corner to corner . . . 16
11.3 Test 3 — Short path, same row . . . 16
11.4 Test 4 — Must navigate around obstacles (horizontal crossing) . . . 16
11.5 Test 5 — Start on an obstacle (should fail) . . . 17
11.6 Test 6 — End on an obstacle (should fail) . . . 17
11.7 Test 7 — Same start and end cell . . . 17
11.8 Test 8 — Out of bounds coordinates (should fail) . . . 17

11.9 Test 9 — Adjacent cells (one step) 17
11.10 Test 10 — Reverse corner to corner 18
11.11 Test 11 — CLI test without client node 18

12 Increasing the Grid Size **18**
12.1 In pathfinder_server.py 18
12.2 In pathfinder_client.py 18
12.3 Rebuild after changes 18

13 Complete Communication Flow **19**

14 Common Mistakes & Fixes **20**

15 Quick Reference **20**
15.1 Commands Cheat Sheet 20
15.2 Package Summary 21
15.3 Test Cases Summary 21

1 Introduction to the Service-Client Model

In ROS 2, nodes communicate in several ways. The two most fundamental patterns are **topics** and **services**. Topics follow a publish-subscribe model where a publisher continuously broadcasts data and any number of subscribers can listen — there is no reply, no acknowledgement, and no guarantee that anyone received the message. This makes topics ideal for streaming data like sensor readings, camera frames, or velocity commands.

Services work differently. A service follows a strict **request-response** model, much like a phone call. One node — the **client** — sends a request and then waits. Another node — the **server** — receives that request, processes it, and sends back a response. Only then does the client continue. This two-way, synchronous interaction makes services the right tool whenever you need a definitive answer: "did that command succeed?", "what is the shortest path?", "is the battery sufficient for this mission?"

The service model is particularly powerful in robotics because robots constantly need to make decisions based on computed results. A navigation stack needs to ask: *"given my current position and target, what is the best route?"* A mission planner needs to ask: *"does this robot have enough battery to complete the task?"* These are not continuous data streams — they are discrete, on-demand queries that demand a reply before execution can continue. Services are the ROS 2 mechanism designed precisely for this.

1.1 What This Tutorial Builds

This tutorial walks through building a complete **grid pathfinder** as a ROS 2 service. The robot operates on a 10-column by 10-row grid where some cells are blocked by obstacles. The client node sends a start cell and an end cell. The server node computes the shortest path around all obstacles using the Breadth-First Search algorithm and returns the full sequence of steps back to the client. The client then prints a step-by-step log and draws an ASCII map of the grid showing the computed path.

This is a realistic pattern that mirrors how real robot navigation works — a motion planner queries a path planning service, receives a waypoint sequence, and executes it. By building this from scratch, you will understand how to define custom service types, structure multi-field requests and responses, implement non-trivial logic inside a server callback, and consume results in a clean client node.

2 Why Two Separate Packages?

A common question when starting with ROS 2 services is: *why do we need two packages — one for the interface and one for the nodes?* The answer comes down to how ROS 2 builds and distributes code.

Custom service types (defined in `.srv` files) must be compiled by a code generation tool called `rosidl`. This tool reads the `.srv` definition and automatically generates Python classes and C++ headers that both the server and client can import. This code generation step requires the **ament_cmake** build system, because `rosidl` is a CMake-based tool.

Python node packages, on the other hand, use the **ament_python** build system, which

does not support `rosidl` code generation. You cannot define custom `.srv` files inside an `ament_python` package — the build system simply does not know how to process them.

The clean solution adopted by the entire ROS 2 ecosystem is to **separate interface definitions from node implementations**:

Package	Build Type	Contains
`pathfinder_interfaces`	`ament_cmake`	`.srv` file, `CMakeLists.txt`
`pathfinder_nodes`	`ament_python`	server node, client node, `setup.py`

This separation also has a practical benefit: the interface package can be shared across many different node packages. If you later write a C++ node, a web dashboard, or a simulation tool that all need to call the same pathfinder service, they all simply depend on `pathfinder_interfaces` and import the same generated types. The interface becomes a stable contract between systems.

Rule of Thumb

One `ament_cmake` package for your `.srv`, `.msg`, and `.action` files. One or more `ament_python` (or `ament_cmake`) packages for your actual nodes. Always build the interface package first.

3 The Grid and the Algorithm

3.1 Grid Layout

The robot operates on a **10 column by 10 row** grid. Coordinates are written as `(x, y)` where `x` is the column (0–9, left to right) and `y` is the row (0–9, top to bottom). Cell `(0,0)` is the top-left corner.

3.2 Obstacle Map

The following cells are blocked and cannot be traversed:

```
(2,1)(3,1)(4,1)
(4,2)(8,2)
(4,3)(8,3)
(2,4)(4,4)
(2,5)(4,5)(5,5)(6,5)(7,5)
(4,6)(5,6)(6,6)(7,6)
```

3.3 Breadth-First Search (BFS)

BFS explores the grid layer by layer, starting from the start cell and expanding outward one step at a time. Because every move costs the same (one step), the first time BFS reaches the destination it is guaranteed to have found the **shortest possible path**.

```
Start
  |
  +--> explore all cells 1 step away
         +--> explore all cells 2 steps away
                +--> explore all cells 3 steps away
                       ...
                         +--> reach End  <-- shortest path guaranteed
```

Each cell can move in four directions: UP, DOWN, LEFT, RIGHT (no diagonals).

4 Create the Two Packages

4.1 Navigate to workspace src

```
cd ~/ros2_ws/src
```

4.2 Package 1 — Interface package

```
ros2 pkg create pathfinder_interfaces \
  --build-type ament_cmake \
  --dependencies rosidl_default_generators
```

4.3 Package 2 — Node package

```
ros2 pkg create pathfinder_nodes \
  --build-type ament_python \
  --dependencies rclpy
```

4.4 Expected workspace structure

```
ros2_ws/
 src/
     pathfinder_interfaces/    <- ament_cmake  (srv definition)
        CMakeLists.txt
        package.xml

     pathfinder_nodes/         <- ament_python (server + client)
         pathfinder_nodes/
            __init__.py
         package.xml
         setup.cfg
         setup.py
         resource/
             pathfinder_nodes
```

5 Set Up the Interface Package

5.1 Create the srv folder and file

```
cd ~/ros2_ws/src/pathfinder_interfaces
mkdir srv
touch srv/FindPath.srv
```

5.2 Write the .srv definition

Open `srv/FindPath.srv` and write exactly this — **no comments, no blank lines**:

FindPath.srv

```
int32 start_x
int32 start_y
int32 end_x
int32 end_y
---
bool success
string message
int32[] path_x
int32[] path_y
int32 total_steps
```

No Comments in .srv Files

In some versions of ROS 2 Humble, comment lines (`#`) at the top of a `.srv` file cause a `NameError` during `rosidl` processing. The safest practice is to write `.srv` files with no comment lines at all.

5.3 Update CMakeLists.txt

Replace the entire content of `pathfinder_interfaces/CMakeLists.txt` :

Listing 1: pathfinder_interfaces/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(pathfinder_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/FindPath.srv"
)

ament_package()
```

5.4 Update pathfinder_interfaces/package.xml

Listing 2: pathfinder_interfaces/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>pathfinder_interfaces</name>
  <version>0.0.0</version>
  <description>Service interfaces for the pathfinder</description
    >
  <maintainer email="you@email.com">your_name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <build_depend>rosidl_default_generators</build_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

5.5 Build and verify

```
cd ~/ros2_ws
colcon build --packages-select pathfinder_interfaces
source install/setup.bash
ros2 interface show pathfinder_interfaces/srv/FindPath
```

Expected Output

```
int32 start_x
int32 start_y
int32 end_x
int32 end_y
---
bool success
string message
int32[] path_x
int32[] path_y
int32 total_steps
```

6 Set Up the Node Package

6.1 Update pathfinder_nodes/package.xml

Listing 3: pathfinder_nodes/package.xml

```
<?xml version="1.0"?>
```

```
<package format="3">
  <name>pathfinder_nodes</name>
  <version>0.0.0</version>
  <description>Pathfinder service server and client</description>
  <maintainer email="you@email.com">your_name</maintainer>
  <license>Apache-2.0</license>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>pathfinder_interfaces</exec_depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

6.2 Create the node files

```
cd ~/ros2_ws/src/pathfinder_nodes/pathfinder_nodes
touch pathfinder_server.py
touch pathfinder_client.py
```

7 Write the Server Node

Listing 4: pathfinder_server.py

```python
import rclpy
from rclpy.node import Node
from collections import deque
from pathfinder_interfaces.srv import FindPath


class PathfinderServer(Node):

    COLS = 10
    ROWS = 10

    OBSTACLES = {
        (2,1),(3,1),(4,1),
        (4,2),(8,2),
        (4,3),(8,3),
        (2,4),(4,4),
        (2,5),(4,5),(5,5),(6,5),(7,5),
        (4,6),(5,6),(6,6),(7,6),
    }

```

```python
    def __init__(self):
        super().__init__('pathfinder_server')

        self.srv = self.create_service(
            FindPath,
            'find_path',
            self.find_path_callback
        )

        self.get_logger().info('Pathfinder Server is ready.')
        self.get_logger().info(
            f'Grid: {self.COLS} cols x {self.ROWS} rows'
        )
        self.get_logger().info(
            f'Obstacles: {len(self.OBSTACLES)} cells'
        )

    def is_valid(self, x, y):
        in_bounds = 0 <= x < self.COLS and 0 <= y < self.ROWS
        free      = (x, y) not in self.OBSTACLES
        return in_bounds and free

    def bfs(self, start, end):
        queue   = deque()
        visited = set()
        queue.append([start])
        visited.add(start)

        while queue:
            path    = queue.popleft()
            current = path[-1]

            if current == end:
                return path

            x, y = current
            for dx, dy in [(0,-1),(0,1),(-1,0),(1,0)]:
                nx, ny    = x + dx, y + dy
                neighbour = (nx, ny)
                if self.is_valid(nx, ny) and neighbour not in visited:
                    visited.add(neighbour)
                    queue.append(path + [neighbour])

        return None

    def find_path_callback(self, request, response):

        start = (request.start_x, request.start_y)
        end   = (request.end_x,   request.end_y)

```

```python
        self.get_logger().info(f'Request: {start} --> {end}')

        if not self.is_valid(*start):
            response.success     = False
            response.message     = (
                f'Start {start} is invalid - out of bounds or on obstacle.'
            )
            response.total_steps = 0
            return response

        if not self.is_valid(*end):
            response.success     = False
            response.message     = (
                f'End {end} is invalid - out of bounds or on obstacle.'
            )
            response.total_steps = 0
            return response

        if start == end:
            response.success     = True
            response.message     = 'Already at destination.'
            response.path_x      = [request.start_x]
            response.path_y      = [request.start_y]
            response.total_steps = 0
            return response

        path = self.bfs(start, end)

        if path is None:
            response.success     = False
            response.message     = (
                f'No path found from {start} to {end}.'
            )
            response.total_steps = 0
            return response

        response.success     = True
        response.path_x      = [p[0] for p in path]
        response.path_y      = [p[1] for p in path]
        response.total_steps = len(path) - 1
        response.message     = (
            f'Path found in {response.total_steps} steps.'
        )

        path_str = ' -> '.join([f'({p[0]},{p[1]})' for p in path])
        self.get_logger().info(f'Path: {path_str}')
        self.get_logger().info(f'Steps: {response.total_steps}')

```

```
        return response


def main(args=None):
    rclpy.init(args=args)
    node = PathfinderServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

7.1 Key Parts Explained

Code	Purpose
`OBSTACLES = {(x,y),...}`	Set of blocked cells read from the grid image
`is_valid(x, y)`	Returns True if cell is in bounds AND not an obstacle
`bfs(start, end)`	BFS algorithm — returns full path list or None
`deque()`	Double-ended queue — efficient for BFS
`queue.append([start])`	Each entry in the queue is the full path so far
`for dx, dy in [(0,-1),(0,1),...]`	Explores 4 neighbours: UP, DOWN, LEFT, RIGHT
`response.path_x / path_y`	Arrays of x and y coordinates for each step
`return response`	Always required — client hangs if omitted

8 Write the Client Node

Listing 5: pathfinder_client.py

```
import sys
import rclpy
from rclpy.node import Node
from pathfinder_interfaces.srv import FindPath


class PathfinderClient(Node):

    OBSTACLES = {
        (2,1),(3,1),(4,1),
        (4,2),(8,2),
        (4,3),(8,3),
        (2,4),(4,4),
        (2,5),(4,5),(5,5),(6,5),(7,5),
        (4,6),(5,6),(6,6),(7,6),
    }

```

```python
    def __init__(self):
        super().__init__('pathfinder_client')

        self.client = self.create_client(
            FindPath,
            'find_path'
        )

        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for pathfinder server ...')

        self.get_logger().info('Pathfinder server connected.')

    def find_path(self, start_x, start_y, end_x, end_y):
        request          = FindPath.Request()
        request.start_x = start_x
        request.start_y = start_y
        request.end_x   = end_x
        request.end_y   = end_y

        self.future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, self.future)
        return self.future.result()

    def draw_grid(self, path_cells, start, end, cols=10, rows=10):

        path_set = set(zip(path_cells[0], path_cells[1]))

        print()
        print('     ' + '  '.join([str(x) for x in range(cols)]))
        print('   +' + ('--+' * cols))

        for y in range(rows):
            row = f' {y} |'
            for x in range(cols):
                cell = (x, y)
                if cell == start:
                    row += 'S |'
                elif cell == end:
                    row += 'E |'
                elif cell in self.OBSTACLES:
                    row += '##|'
                elif cell in path_set:
                    row += '. |'
                else:
                    row += '  |'
            print(row)
            print('   +' + ('--+' * cols))

```

```
        print()
        print('  S=Start  E=End  .=Path  ##=Obstacle')
        print()


def main(args=None):
    rclpy.init(args=args)
    node = PathfinderClient()

    start_x = int(sys.argv[1])
    start_y = int(sys.argv[2])
    end_x   = int(sys.argv[3])
    end_y   = int(sys.argv[4])

    result = node.find_path(start_x, start_y, end_x, end_y)

    print()
    print('========================================')
    print(f'  From      : ({start_x}, {start_y})')
    print(f'  To        : ({end_x},   {end_y})')
    print('========================================')

    if result.success:
        print(f'  Status    : PATH FOUND')
        print(f'  Steps     : {result.total_steps}')
        print('========================================')
        print('  Step-by-step:')

        for i, (x, y) in enumerate(zip(result.path_x, result.
            path_y)):
            if i == 0:
                print(f'    Step {i:2d}: ({x},{y})  <- START')
            elif i == len(result.path_x) - 1:
                print(f'    Step {i:2d}: ({x},{y})  <- END')
            else:
                print(f'    Step {i:2d}: ({x},{y})')

        print('========================================')
        node.draw_grid(
            path_cells = (result.path_x, result.path_y),
            start      = (start_x, start_y),
            end        = (end_x, end_y)
        )
    else:
        print(f'  Status    : NO PATH FOUND')
        print(f'  Reason    : {result.message}')
        print('========================================')

    node.destroy_node()
    rclpy.shutdown()

```

```python

if __name__ == '__main__':
    main()
```

9 Update setup.py

Listing 6: pathfinder_nodes/setup.py

```python
from setuptools import find_packages, setup

package_name = 'pathfinder_nodes'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='your_name',
    maintainer_email='you@email.com',
    description='Pathfinder service server and client',
    license='Apache-2.0',
    extras_require={
        'test': ['pytest'],
    },
    entry_points={
        'console_scripts': [
            'pathfinder_server = pathfinder_nodes.
              pathfinder_server:main',
            'pathfinder_client = pathfinder_nodes.
              pathfinder_client:main',
        ],
    },
)
```

10 Build and Run

10.1 Build both packages

```bash
cd ~/ros2_ws

# Step 1 -- interfaces first (nodes depend on them)
colcon build --packages-select pathfinder_interfaces
source install/setup.bash
```

```
# Step 2 -- then the nodes
colcon build --packages-select pathfinder_nodes
source install/setup.bash
```

Expected Build Output

```
Starting >>> pathfinder_interfaces
Finished <<< pathfinder_interfaces [6.62s]
Summary: 1 package finished

Starting >>> pathfinder_nodes
Finished <<< pathfinder_nodes [1.73s]
Summary: 1 package finished
```

10.2 Terminal 1 — Start the server

```
ros2 run pathfinder_nodes pathfinder_server
```

Server Startup Output

```
[INFO] [pathfinder_server]: Pathfinder Server is ready.
[INFO] [pathfinder_server]: Grid: 10 cols x 10 rows
[INFO] [pathfinder_server]: Obstacles: 18 cells
```

10.3 Terminal 2 — Run the client

```
# Usage: pathfinder_client <start_x> <start_y> <end_x> <end_y>
ros2 run pathfinder_nodes pathfinder_client 1 1 8 6
```

11 Test Cases

Run the server in Terminal 1, then run each test in Terminal 2.

11.1 Test 1 — Standard path (original grid example)

```
ros2 run pathfinder_nodes pathfinder_client 1 1 8 6
```

What to expect

Robot starts at (1,1) and must reach (8,6). It must navigate around the horizontal bar obstacle at y=1, the vertical bar at x=4, and the bottom block obstacles. BFS finds the shortest route going down the left side and across the bottom.

Expected Output

```
From      : (1, 1)
To        : (8,   6)
Status    : PATH FOUND
Steps     : (varies based on grid)

    0  1  2  3  4  5  6  7  8  9
  +--+--+--+--+--+--+--+--+--+--+
0 |  |  |  |  |  |  |  |  |  |  |
  +--+--+--+--+--+--+--+--+--+--+
1 |  |S |##|##|##|  |  |  |  |  |
  +--+--+--+--+--+--+--+--+--+--+
2 |  |. |  |  |##|  |  |  |##|  |
  ...
S=Start  E=End  .=Path  ##=Obstacle
```

11.2 Test 2 — Corner to corner

```
ros2 run pathfinder_nodes pathfinder_client 0 0 9 9
```

What to expect

Longest possible path on the grid. Robot travels from top-left to bottom-right, navigating all obstacles in between.

11.3 Test 3 — Short path, same row

```
ros2 run pathfinder_nodes pathfinder_client 0 0 5 0
```

What to expect

Start and end are on the same row with no obstacles between them. BFS finds a straight horizontal path of exactly 5 steps.

Expected Output

```
Steps     : 5
Step  0: (0,0)  <- START
Step  1: (1,0)
Step  2: (2,0)
Step  3: (3,0)
Step  4: (4,0)
Step  5: (5,0)  <- END
```

11.4 Test 4 — Must navigate around obstacles (horizontal crossing)

```
ros2 run pathfinder_nodes pathfinder_client 0 3 9 3
```

What to expect

Start and end are on the same row but obstacle at (4,3) and (8,3) blocks the straight path. BFS must route either above or below the obstacles.

11.5 Test 5 — Start on an obstacle (should fail)

```
ros2 run pathfinder_nodes pathfinder_client 2 1 8 6
```

Expected Output

```
Status   : NO PATH FOUND
Reason   : Start (2, 1) is invalid - out of bounds or on obstacle.
```

11.6 Test 6 — End on an obstacle (should fail)

```
ros2 run pathfinder_nodes pathfinder_client 0 0 4 5
```

Expected Output

```
Status   : NO PATH FOUND
Reason   : End (4, 5) is invalid - out of bounds or on obstacle.
```

11.7 Test 7 — Same start and end cell

```
ros2 run pathfinder_nodes pathfinder_client 3 3 3 3
```

Expected Output

```
Status   : PATH FOUND
Steps    : 0
Message  : Already at destination.
```

11.8 Test 8 — Out of bounds coordinates (should fail)

```
ros2 run pathfinder_nodes pathfinder_client 0 0 15 15
```

Expected Output

```
Status   : NO PATH FOUND
Reason   : End (15, 15) is invalid - out of bounds or on obstacle.
```

11.9 Test 9 — Adjacent cells (one step)

```
ros2 run pathfinder_nodes pathfinder_client 0 0 1 0
```

Expected Output

```
Steps    : 1
Step  0: (0,0)  <- START
Step  1: (1,0)  <- END
```

11.10 Test 10 — Reverse corner to corner

```
ros2 run pathfinder_nodes pathfinder_client 9 9 0 0
```

What to expect

Same path as Test 2 but in reverse. BFS computes this independently and returns a path from bottom-right to top-left.

11.11 Test 11 — CLI test without client node

You can also call the service directly from the terminal without running the client node at all:

```
ros2 service call /find_path \
  pathfinder_interfaces/srv/FindPath \
  "{start_x: 1, start_y: 1, end_x: 8, end_y: 6}"
```

12 Increasing the Grid Size

To change the grid to any size, only two values need updating.

12.1 In pathfinder_server.py

```
# Change ROWS to any number
COLS = 10
ROWS = 10    # <- change this to 15, 20, etc.
```

12.2 In pathfinder_client.py

```
# Change the rows default argument
def draw_grid(self, path_cells, start, end, cols=10, rows=10):
#                                                          ^^
#                                          change to match
    server
```

12.3 Rebuild after changes

```
colcon build --packages-select pathfinder_nodes
source install/setup.bash
```

13 Complete Communication Flow

```
Terminal command:
ros2 run pathfinder_nodes pathfinder_client 1 1 8 6
          |
          v
main()
  rclpy.init()
  node = PathfinderClient()
    create_client('find_path')
    wait_for_service() -- server found
          |
          v
  start_x=1, start_y=1, end_x=8, end_y=6
          |
          v
  find_path(1, 1, 8, 6)
    request.start_x = 1  request.start_y = 1
    request.end_x   = 8  request.end_y   = 6
    call_async(request) -----------------> PathfinderServer
    spin_until_future_complete()             find_path_callback()
    future.result() <-----------------        is_valid(1,1) -> True
          |                                   is_valid(8,6) -> True
          v                                   bfs((1,1),(8,6))
  result.success  = True                        -> path found
  result.total_steps = N                      return response
  result.path_x  = [1,0,0,0,...]
  result.path_y  = [1,1,2,3,...]
          |
          v
  print step-by-step path
  draw_grid() -- ASCII visualisation
  destroy_node()
  rclpy.shutdown()
  exits
```

14 Common Mistakes & Fixes

Mistake	Error	Fix
Comments in `.srv` file	`NameError: invalid field name`	Remove all `#` lines from `.srv`
Typo in `ros2 pkg create`	`ignoring unknown package`	Check spelling exactly
Build node package before interface	Import error	Always build interfaces first
Forgot to source after build	`Package not found`	`source install/setup.bash`
Obstacle includes valid start cell	`Start is invalid`	Check `OBSTACLES` set carefully
Forgot `return response`	Client hangs forever	Always `return response` in callback
Wrong argument count to client	`IndexError`	Pass all 4 args: `x1 y1 x2 y2`
`OBSTACLES` mismatch between server and client	Grid drawn incorrectly	Keep both sets identical

15 Quick Reference

15.1 Commands Cheat Sheet

```
# Build a specific package
colcon build --packages-select <package_name>

# Source after every build
source install/setup.bash

# Run server
ros2 run pathfinder_nodes pathfinder_server

# Run client
ros2 run pathfinder_nodes pathfinder_client <x1> <y1> <x2> <y2>

# List all active services
ros2 service list

# Call service from terminal directly
ros2 service call /find_path \
  pathfinder_interfaces/srv/FindPath \
  "{start_x: 0, start_y: 0, end_x: 9, end_y: 9}"

# Show service type definition
ros2 interface show pathfinder_interfaces/srv/FindPath
```

15.2 Package Summary

Package	Type	Contents
`pathfinder_interfaces`	`ament_cmake`	`FindPath.srv`
`pathfinder_nodes`	`ament_python`	`pathfinder_server.py`, `pathfinder_client.py`

15.3 Test Cases Summary

#	Start	End	Expected	Tests
1	(1,1)	(8,6)	PATH FOUND	Standard navigation
2	(0,0)	(9,9)	PATH FOUND	Corner to corner
3	(0,0)	(5,0)	PATH FOUND, 5 steps	Straight line
4	(0,3)	(9,3)	PATH FOUND	Around obstacles
5	(2,1)	(8,6)	NO PATH	Start on obstacle
6	(0,0)	(4,5)	NO PATH	End on obstacle
7	(3,3)	(3,3)	0 steps	Same cell
8	(0,0)	(15,15)	NO PATH	Out of bounds
9	(0,0)	(1,0)	PATH FOUND, 1 step	Adjacent cells
10	(9,9)	(0,0)	PATH FOUND	Reverse corner
11	(1,1)	(8,6)	PATH FOUND	CLI call, no client node

End of Tutorial — ROS 2 Pathfinder Service & Client